

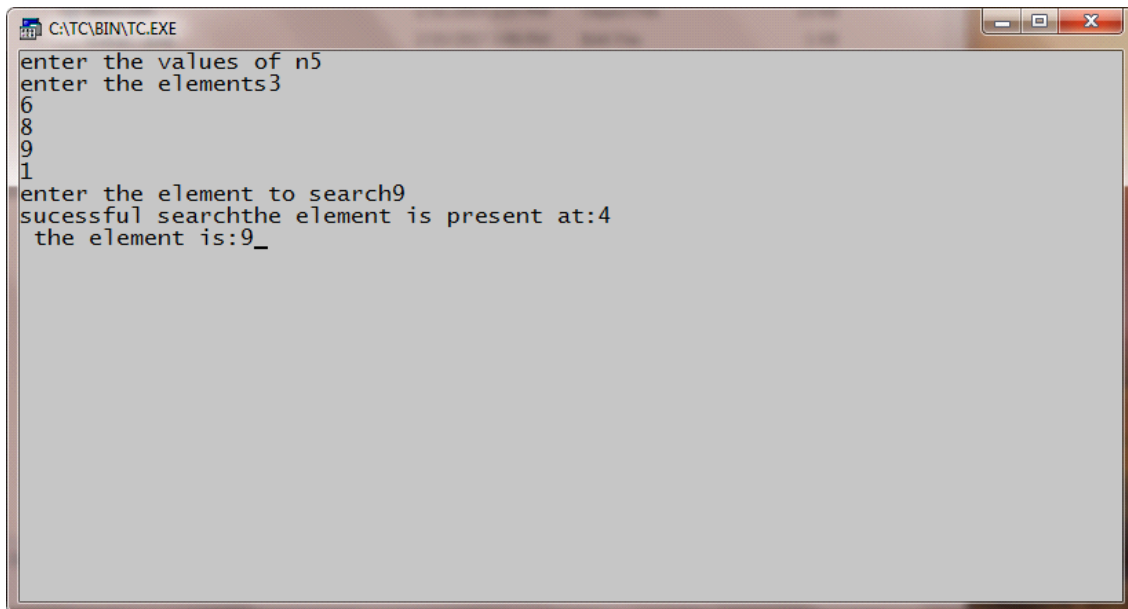
**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

BINARY SEARCH

```
#include<iostream>
using namespace std;
int main()
{
    int a[30],x,k,l,m,n,h;
    cout<<"Enter the values of n";
    cin>>n;
    cout<<"Enter the elements";
    for(k=0;k<n;k++)
        cin>>a[k];
    cout<<"Enter the element to search";
    cin>>x;
    l=0;
    h=n-1;
    while(l<=h)
    {
        m=(l+h)/2;
        if(x==a[m])
        {
            cout<<"\nSuccessful search\n";
            cout<<"\nThe element is present"<<m+1;
            cout<<"\nThe element is:"<<x;
            break;
        }
        if(x<a[m])
        {
            h=m-1;
            continue;
        }
        if(x>a[m])
        {
            l=m+1;
            continue;
        }
    }
    while(l>=h)
    {
        cout<<"Unsuccessful search";
    }
    return 0;
```

}

OUTPUT:



A screenshot of a Turbo C++ console window titled "C:\TC\BIN\TC.EXE". The window contains the following text:

```
enter the values of n5  
enter the elements3  
6  
8  
9  
1  
enter the element to search9  
sucessful searchthe element is present at:4  
the element is:9_
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

MERGE SORT

```
#include<iostream>
using namespace std;
void merge (int[],int,int,int);
void part(int[],int,int);
int main()
{
    int arr[30];
    int i, size;
    cout<<"\n\t-----merge sorting method-----\n\n";
    cout<<"\nEnter total no of elements:";
    cin>>size;
    for(i=0;i<size;i++)
    {
        cout<<"\nEnter"<<i+1<<"element:";
        cin>>arr[i];
    }
    part (arr,0,size-1);
    cout<<"\n\n----merge sorted elements-----\n\n";
    for(i=0;i<size;i++)
        cout<<"\n"<<arr[i];
    return 0;
}
void part(int arr[],int min,int max)
{
    int mid;
    if(min<max)
    {

        mid=(min+max)/2;
        part(arr,min,mid);
        part(arr,mid+1,max);
        merge(arr,min,mid,max);

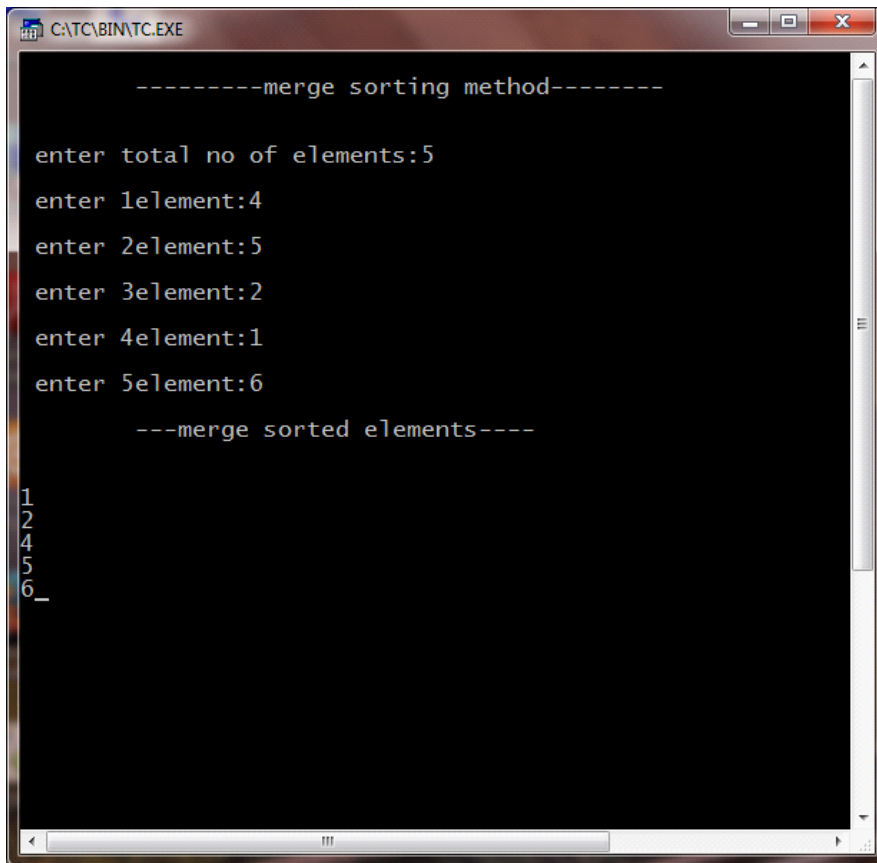
    }
}
void merge(int arr[],int min,int mid, int max)
{
    int temp[30];
    int i,j,k,m;
```

```

j=min;
m=mid+1;
for(i=min;j<=mid&& m<=max;i++)
{
    if(arr[j]<=arr[m])
    {
        temp[i]=arr[j];
        j++;
    }
    else
    {
        temp[i]=arr[m];
        m++;
    }
}
if(j>mid)
{
    for(k=m;k<=max;k++)
    {
        temp[i]=arr[k];
        i++;
    }
}
else
{
    for(k=j;k<=mid;k++)
    {
        temp[i]=arr[k];
        i++;
    }
}
for(k=min;k<=max;k++)
    arr[k]=temp[k];
}

```

OUTPUT:



```
-----merge sorting method-----  
  
enter total no of elements:5  
enter 1element:4  
enter 2element:5  
enter 3element:2  
enter 4element:1  
enter 5element:6  
  
---merge sorted elements---  
  
1  
2  
4  
5  
6_  
_
```

**NAME: MOHAMED B SIRAJUDDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

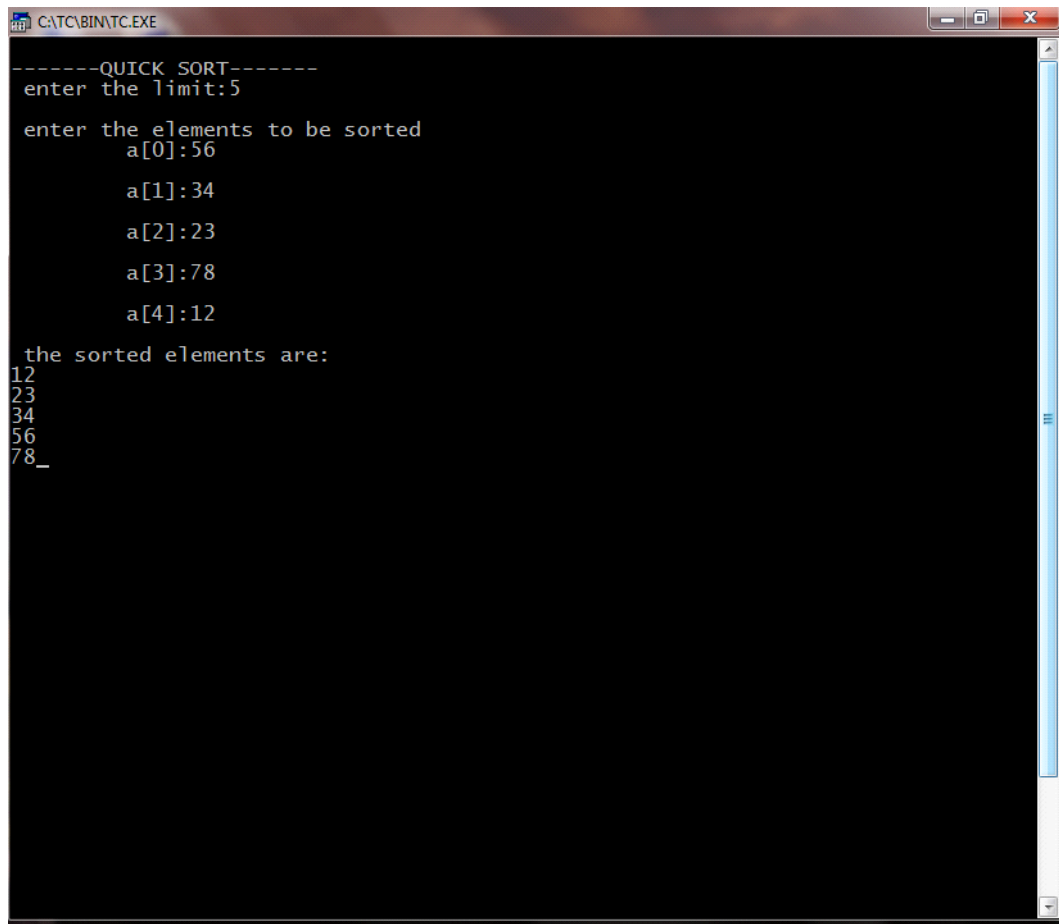
QUICK SORT

```
#include<iostream>
using namespace std;
void quick(int a[], int,int);
void split (int a[],int,int);
int main()
{
    int i,n,a[20];
    cout<<"\nEnter the limit:";
    cin>>n;
    cout<<"\nEnter the elements to be sorted";
    for(i=0;i<n;i++)
    {
        cout<<"\n\t a["<<i<<";
        cin>>a[i];
    }
    quick(a,0,n-1);
    cout <<"\nThe sorted elements are:";
    for(i=0;i<n;i++)
        cout<<"\n"<<a[i];
}
void quick(int a[],int first, int last)
{
    int x,i,j;
    if(first<last)
    {
        x=a[first];
        i=first;
        j=last;
        while(i<j)
        {
            while(a[i]<=x&&i<last)
                i++;
            while(a[j]>=x&&j>first)
                j--;
            if(i<j)
                split(a,i,j);
        }
    }
}
```

```
        split (a,first,j);
        quick(a,first,j-1);
        quick(a,j+1,last);
    }
}
void split(int a[],int i,int j)
{

    int temp;
    temp=a[i];
    a[i]=a[j];
    a[j]=temp;
}
```

OUTPUT:



```
-----QUICK SORT-----
enter the limit:5

enter the elements to be sorted
  a[0]:56
    a[1]:34
      a[2]:23
        a[3]:78
          a[4]:12

the sorted elements are:
12
23
34
56
78_
```


**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

KNAP SACK

```
#include<iostream>
using namespace std;
float w[100],v[100],x[100],wt,wgt[100];
class greed
{
private:
    int i,j,n;
    float r[100],f,total,t,rat,pro,wei,op,opti;
public:
    void getdata();
    void ratio();
    void profit();
    void weight();
    void optimal();
};
void greedy(int n)
{
    int wgt,i;
    for(i=0;i<n;i++)
        x[i]=0;
    wgt=i=0;
    while(wgt<wt)
    {
        if(wgt+w[i]<=wt)
        {
            x[i]=1;
            wgt+=w[i];
        }
        else
        {
            x[i]=(wt-wgt)/w[i];
            wgt+=(x[i]*w[i]);
        }
        i++;
    }
}
```

```

}
}
void greed::getdata()
{
    cout<<"\nEnter no of objects _____";
    cin>>n;
    cout<<"\nEnter the capacity of the bag _____";
    cin>>wt;
    for(i=0;i<n;i++)
    {
        cout<<"weight=";
        cin>>w[i];
        cout<<"profit=";
        cin>>v[i];
        r[i]=v[i]/w[i];

    }

}

void greed::ratio()
{

    cout<<"GREEDY BY RATIO2
    2222\n";
    for(i=0;i<n;i++)
        for(j=i+1;j<n;j++)
            if(r[i]<r[j])
            {
                t=w[i];w[i]=w[j];w[j]=t;
                t=v[i];v[i]=v[j];v[j]=t;
                t=r[i];r[i]=r[j];r[j]=t;

            }
    greedy(n);
    for(i=0;i<n;i++)
    {
        cout<<"\n weight="<<w[i];
        cout<<"\n profit="<<v[i];
        cout<<"\n ratio="<<r[i];
        cout<<"\n fraction+"<<x[i];
        total+=(x[i]*v[i]);
    }
    cout<<"\n feasible profit="<<total;
    rat=total;
    total=0;
}

```

```

void greed::profit()
{
    cout<<"\n GREEDY BY PROFIT";
    for(i=0;i<n;i++)
        for(j=i+1;j<n;j++)
            if(v[i]<v[j])
            {
                t=w[i];w[i]=w[j];w[j]=t;
                t=v[i];v[i]=v[j];v[j]=t;
                t=r[i];r[i]=r[j];r[j]=t;
            }
    greedy(n);
    for(i=0;i<n;i++)
    {
        cout<<"\n weight="<<w[i];
        cout<<"\n profit="<<v[i];
        cout<<"\n ratio="<<r[i];
        cout<<"\n fraction="<<x[i];
        total+=(x[i]*v[i]);
    }
    cout<<"\n feasible profit="<<total;
    pro=total;
    total=0;
}

void greed::weight()
{
    cout<<"GREEDY BY WEIGHT";
    for(i=0;i<n;i++)
        for(j=j+1;j<n;j++)
            if(w[i]<w[j])
            {
                t=w[i];w[i]=w[j];w[j]=t;
                t=v[i];v[i]=v[j];v[j]=t;
                t=r[i];r[i]=r[j];r[j]=t;
            }
    greedy(n);
    for(i=0;i<n;i++)
    {
        cout<<"\n weight="<<w[i];
        cout<<"\n profit="<<v[i];
        cout<<"\n ratio="<<r[i];
        cout<<"\n fraction="<<x[i];
        total+=(x[i]*v[i]);
    }
    cout<<"\n feasible profit="<<total;
    wei=total;
}

```

```

}
void greed::optimal()
{
    if(rat>pro)
        op=rat;
    else
        op=pro;
    if(op>wei)
        opti=op;
    else
        opti=wei;
    cout<<"\n optimal solutions....."<<opti;
}
int main()
{
    greed s;
    cout<<"\nKNAPSACK PROBLEM";
    s.getdata();
    s.ratio();
    s.profit();
    s.weight();
    s.optimal();
}

```

OUTPUT:

```
CATC\BIN\TC.EXE

KNAP SACK PROBLEM..
enter no of object----2

enter the capacity of the bag--20
weight=40
profit=10
weight=55
profit=15
GREEDY BY RATION
weight=40
profit=15
ratio=0.272727
fraction=0.5
weight=40
profit=10
ratio=0.25
fraction=0
feasible profit=7.5
GREEDY BY PROFIT
weight=40
profit=15
ratio =0.272727
fraction=0.5
weight=40
profit=10
ratio =0.25
fraction=0
feasible profit=7.5
GREEDY BY WEIGHT
weight=40
profit=15
ratio=0.272727
fraction=0.5
weight=40
profit=10
ratio=0.25
fraction=0
feasible profit=7.5
optimal solution is .....7.5_
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

SINGLE SOURCE SHORTEST PATH

```
#include<iostream>
using namespace std;
class path
{
    private:
        int i,j,m,n,b[50],cost[20][20],k,d[50];
        int u,v,w,num,s[20];
    public:
        void getdata();
        void sp();
        void display();
};
void path::getdata()
{
    cout<<"\n***SINGLE SOURCE SHORTEST PATH***\n";
    cout<<"\n enter the number of nodes:";
    cin>>n;
    cout<<"\n enter the cost\n";
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            if(i==j)
            {
                cost[i][j]=0;
            }
            else
            {
                cout<<"\n cost["<<i<<","<<j<<"]:";
                cin>>cost[i][j];
            }
        }
    }
    cout<<"\n";
```

```

    }
}
void path::sp()
{
    k=1;
    cout<<"\n the cost of vertices are:\n";
    cout<<"\t";
    for(i=1;i<=n;i++)
        cout<<i<<"\t";
    for(i=1;j<=n;i++)
        cout<<"-----";
    cout<<"\n";
    for(i=1;i<=n;i++)
    {
        cout<<i<<"\t";
        for(j=1;j<=n;j++)
        {
            cout<<cost[i][j]<<"\t";
        }
    }
    cout<<"\n";
    }
    cout<<"\n enter the source vertex:";
    cin>>v;
    for(i=1;i<=n;i++)
    {
        s[i]=0;
        d[i]=cost [v][i];
    }
    s[v]=1;
    b[k++]=v;
    d[v]=0;
    for(num=2;num<=n;num++)
    {
        m=9999;
        for(j=1;j<=n;j++)
            if((m>d[j])&&(s[j]==0))
            {
                m=d[j];
                u=j;
            }
        s[u]=1;
        b[k+1]=u;
        for(w=1;w<=n;w++)
        {
            if(d[w]>(d[u]+cost[u][w])&&(s[w]==0))
            {

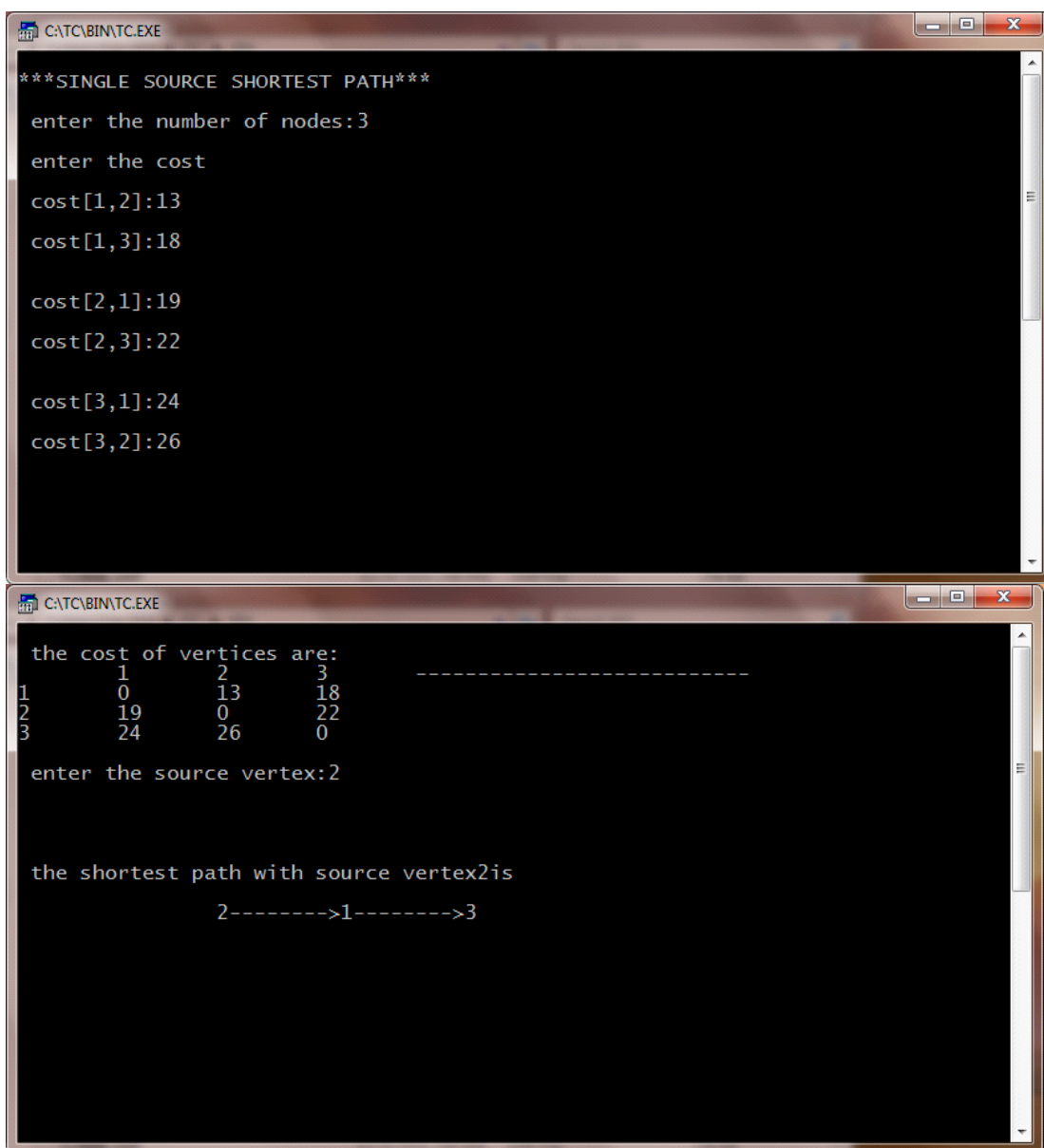
```

```

        d[w]=(d[u]+cost[u][w]);
    }
}
cout<<"\n";
}
}
void path::display()
{
    cout<<"\n";
    cout<<"\n the shortest path with source vertex"<<v<<"is\n";
    cout<<"\n\t\t"<<b[i];
    for(i=2;i<k;i++)
    cout<<"-----"<<b[i];
}
int main()
{
    path p;
    p.getdata();
    p.sp();
    p.display();
}

```


OUTPUT :



The image shows two screenshots of a Windows command prompt window titled "C:\TC\BIN\TC.EXE".

The first screenshot displays the program's output for finding the shortest path. It starts with the title "***SINGLE SOURCE SHORTEST PATH***". It then prompts the user to "enter the number of nodes:3". Next, it prompts the user to "enter the cost". The user enters the following costs:

- cost[1,2]:13
- cost[1,3]:18
- cost[2,1]:19
- cost[2,3]:22
- cost[3,1]:24
- cost[3,2]:26

The second screenshot displays the program's output for finding the shortest path with source vertex 2. It starts with the title "the cost of vertices are:". It then displays a table of costs:

	1	2	3
1	0	13	18
2	19	0	22
3	24	26	0

After the table, it prompts the user to "enter the source vertex:2". It then displays the shortest path with source vertex 2:

the shortest path with source vertex2is
2----->1----->3

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

MULTISTAGE GRAPH

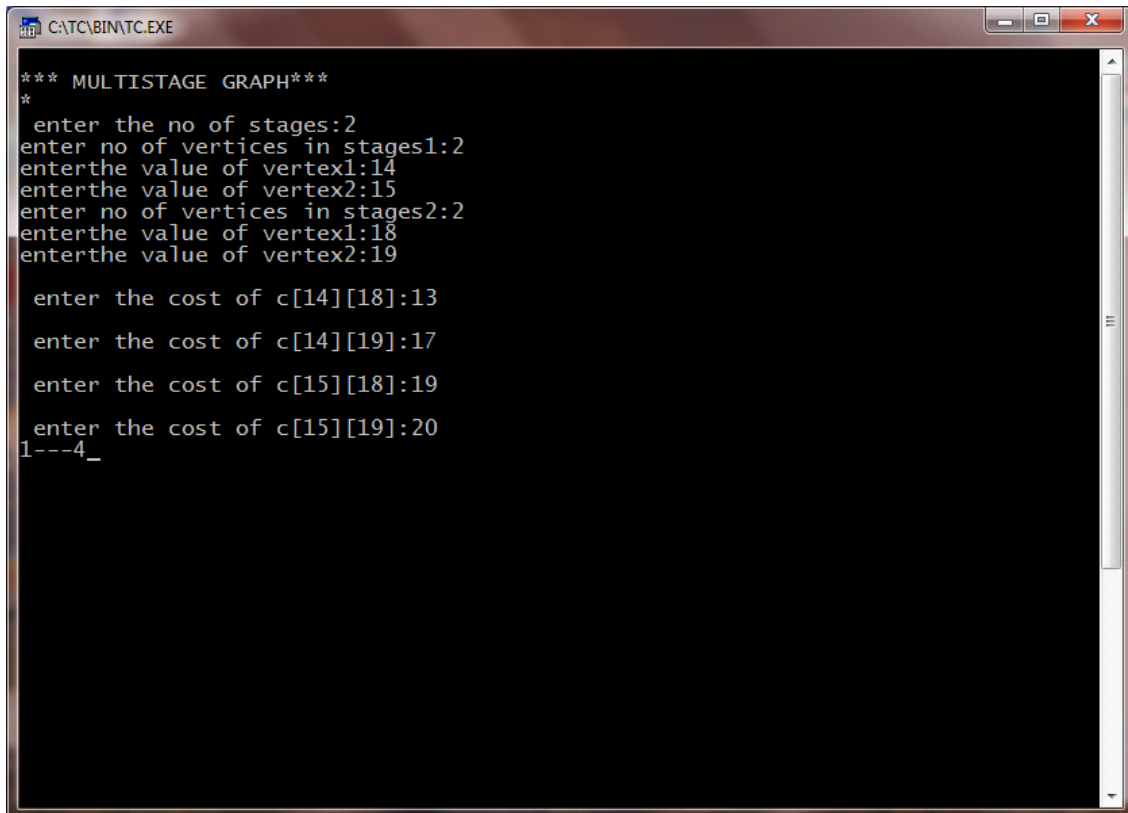
```
#include<iostream>
using namespace std;
struct fwd
{
    int l;
    int a[20];
};
struct fwd s1[10];
int main()
{
    int k,i,j,n,p[10],m,z,cost[50],v,c[20][20];
    cout<<"Enter the number of stages"<<i<<":";
    cin>>k;
    n=0;
    for(i=1;i<=k;i++)
    {
        cout<<"no of vertices of the stage"<<i<<"i";
        cin>>s1[i].l;
        n+=s1[i].l;
        for(j=1;j<=s1[i].l;j++)
        {
            cout<<"Enter the value of vertex"<<j<<":";
            cin>>s1[i].a[j];
        }
    }
    for(i=1;i<k;i++)
    {
        for(j=s1[i].a[1];j<=s1[i].a[s1[i].l];j++)
        {
            for(z=s1[i+1].a[1];z<=s1[i+1].a[s1[i+1].l];z++)
            {
                cout<<"\n enter the cost of c ["<<j<<"]["<<z<<"]";
                cin>>c[j][z];
            }
        }
    }
}
```

```

    }
}
cost[n]=0;
int min,d[50],t;
for(i<=k-1;i>=1;i--)
{
    for(j=s1[i].a[i];j<=s1[i].a[s1[i].l];j++)
    {
        min=999;
        for(z=s1[i+1].a[1];z<=s1[i+1].a[s1[i].l];z++)
        {
            if(cost[z]+c[j][z<min])
                min=cost[z]+c[j][z];
            t=z;
        }
        cost[j]=min;
        d[j]=t;
    }
}
p[1]=1;
p[k]=n;
for(i=2;i<k;i++)
{
    p[i]=d[p[i-1]];
}
for(i=1;i<k;i++)
{
    cout<<p[i]<<".....";
}
cout<<p[k];
return 0;
}

```

OUTPUT:



```
C:\TC\BIN\TC.EXE
*** MULTISTAGE GRAPH***
*
enter the no of stages:2
enter no of vertices in stages1:2
enterthe value of vertex1:14
enterthe value of vertex2:15
enter no of vertices in stages2:2
enterthe value of vertex1:18
enterthe value of vertex2:19

enter the cost of c[14][18]:13
enter the cost of c[14][19]:17
enter the cost of c[15][18]:19
enter the cost of c[15][19]:20
1---4_
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

0/1 KNAP SACK PROBLEM

```
#include<iostream>
using namespace std;
int c,z[20],t[50][50],s[20],i,j,pt[20],wt[20],x[20],m,n,w;
class knp
{
public:
    void get();
    void table();
    void select();
    void display();
}
k;
void knp::get()
{
    cout<<"\n\t0\1 KNAPSACK PROBLEM....";
    cout<<"\nENTER THE CAPACITY OF THE BAG...";
    cin>>m;
    cout<<"\n enter the no of objects...";
    cin>>n;
    for(i=1;i<=n;i++)
    {
        cout<<"\nEnter the weight of objects...";
        cin>>wt[i];
        cout<<"\nEnter the profit of object"<<i<<"....";
        cin>>pt[i];
    }
}
void knp::table()
{
    for(i=1;i<=n;i++)
    {
        for(j=0;j<=m;j++)
```

```

    {
        if((i==1)&&(j<wt[i]))
            t[i][j]=0;
        if((i==1)&&(j>=wt[i]))
            t[i][j]=pt[i];
        if((i>1)&&(j<wt[i]))
            t[i][j]=t[i-1][j];
        if((i>1)&&(j>=wt[i]))
            t[i][j]=pt[i]+t[i-1][j-wt[i]];
    }
}
cout<<"\n\t\t table\n";
for(i=1;i<=n;i++)
{
    for(j=0;j<=m;j++)
        cout<<t[i][j]<<"\t";
    cout<<"\n";
}
}
void knp::select()
{
    int q=0;
    c=0;
    int a,b,d;
    i=n;
    for(j=m;j>=0;j--)
    {
        if(t[i][j]==t[i-1][j])
        {
            a=i-1;
            if(t[a][j]==t[a-1][j-wt[a]]+pt[a])
            {

            }
            else
            {
                z[c]=t[a][j];
                c++;
                s[q]=a;
                q++;
            }
        }
        b=a-1;
        d=j-wt[a];
        if(t[b-1][d]==t[b-1][d-wt[b]])
        {

```

```

    }
    else
    {
        z[c]=t[a-1][j-wt[a]];
        c++;
        s[q]=a-1;
        q++;

    }
}
else
{
    a=i-1;
    if(t[a][j]==t[a-1][j-wt[a]]+pt[a])
    {

    }
    else
    {
        z[c]=t[a][j];
        s[q]=a;
        q++;

    }
    b=a-1;
    d=j-wt[a];
    if(t[a-1][d]==t[b-1][d-wt[b]])
    {

    }
    else
    {
        z[c]=t[a-1][j-wt[a]];
        c++;
        s[q]=a-1;
        q++;

    }
    b=a-1;
    d=j-wt[a];
    if(t[a-1][d]==t[b-1][d-wt[b]])
    {

    }
    else
    {
        z[c]=t[a-1][j-wt[a]];

```

```

        c++;
        s[q]=a-1;
        q++;
    }
}
j=d+1;
}
}
void knp::display()
{
    cout<<"\nThe objects selected are...";
    for(int x=0;x<c;x++)
    {
        cout<<"\t"<<s[x];
    }
}
int main()
{
    char ch='y';
    while(ch=='y')
    {
        k.get();
        k.table();
        k.select();
        k.display();
        cout<<"\n Do you want to continue(y\n)";
        cin>>ch;
    }
}

```


OUTPUT:

```
C:\TC\BIN\TC.EXE
09 KNAPSACK PROBLEM..
enter the capacity of the bag---20
enter the no of objects--2
enter the weight of object1...12
enter the profit of object1.....14
enter the weight of object2...15
enter the profit of object2.....17

      table
0      0      0      0      0      0      0      0      0
0      0      14     14     14     14     14     14     14
14     0      0      0      0      0      0      0      0
0      0      14     14     14     17     17     17     17
17     0      0      0      0      0      0      0      0

the objects selected are--- 0      1      0
do you want to continue(y
)
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

ALL PAIRS SHORTEST PATH

```
#include<iostream>
using namespace std;
int a[20][20],cost[20][20],i,j,n,k;
class path
{
public:
    void get();
    void allpair (int a[20][20],int n);
    void put();
};
void path::get()
{
    cout<<"\n\t ALL PAIR SHORTEST PATH\n";
    cout<<"\n Enter the no of vertex";
    cin>>n;
    cout<<"\nEnter the cost";
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            if(i==j)
            {
                cost[i][j]=0;
                a[i][j]=cost[i][j];
            }
            else
            {
                cout<<"\n cost["<<i<<"]["<<j<<"]="";
                cin>>cost[i][j];
                a[i][j]=cost[i][j];
            }
        }
    }
}
```

```

    }
}
cout<<"\n";

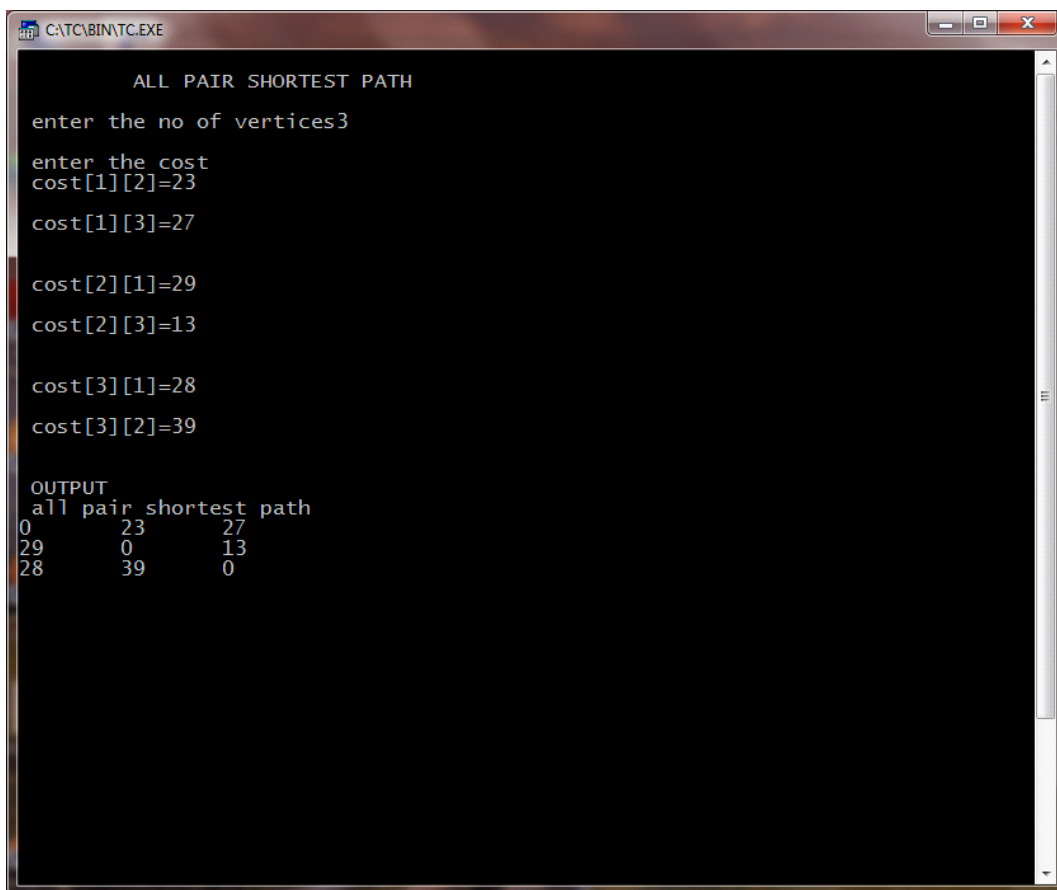
}
}
void path::allpair (int a[20][20],int n)
{
    for(k=1;k<=n;k++)
    {
        for(i=1;i<=n;i++)
        {
            for(j=1;j<=n;j++)
            {
                if((a[i][k]+a[k][j])<a[i][j])
                {
                    a[i][j]=a[i][k]+a[k][j];
                }
            }
        }
    }
}
void path::put()
{

    cout<<"\n OUTPUT";
    cout<<"\n all pair shortest path \n";
    for(i=1;i<=n;i++)
    {
        for(j=1;j<=n;j++)
        {
            cout<<a[i][j];
            cout<<"\t";

        }
        cout<<"\n";
    }
}
int main()
{
    path p;
    p.get();
    p.allpair (a,n);
    p.put();
}

```

OUTPUT:



```
ALL PAIR SHORTEST PATH
enter the no of vertices3
enter the cost
cost[1][2]=23
cost[1][3]=27
cost[2][1]=29
cost[2][3]=13
cost[3][1]=28
cost[3][2]=39

OUTPUT
all pair shortest path
0      23      27
29     0      13
28     39     0
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

TRAVELLING SALESMAN USING DYNAMIC PROGRAMMING

```
#include<iostream>
using namespace std;
class sales
{
    int cost[20][20];
    int i,a[20],min,result[20],j,x,n;
    public:
    void input();
    int dts(int v,int t[10],int x,int path[20]);
    void display();
};

void sales::input()
{
    cout<<"\n travelling salesman problem \n";
    cout<<"\nEnter the values of n:";
    cin>>n;
    cout<<"\nfrom to distance\n";
    do
    {
        cin>>i;
        i--;
        cin>>j;
        j--;
        if((i== -1)&&(j== -1))
            break;
        cin>>cost[i][j];
    }
}
```

```

while(1);
for(i=1;i<=n;i++)
cost[i][j]=0;
for(i=0;i<n-1;i++)
a[i]=i+1;
min=dts(0,&a[0],n-1,&result[0]);
}
int sales::dts(int v,int t[10],int x, int path[20])
{
    int a[20],j,i,k,min=999,temp[10],y;
    for(i=0;i<=n;i++)
    a[i]=0;
    if(x==0)
    {
        *path =v;
        return (cost[v][0]);
    }
    else if(x==1)
    {
        path[0]=t[0];
        path[1]=0;
        return(cost[v][t[0]]+cost[t[0]][0]);
    }
    else
    {
        for(i=0;i<x;i++)
        {
            k=0;
            for(j=0;j<x;j++)
            {
                if (t[j]!=t[i])
                {
                    a[k]=t[j];
                    k++;
                }
            }
            y=(cost[v][t[i]]+dts(t[i],a,x-1,temp));
            if(min>y)
            {
                path[0]=t[i];
                k=1;
                min=y;
                for(j=0;j<x;j++)
                {
                    path[k]=temp[j];
                    k++;
                }
            }
        }
    }
}

```

```

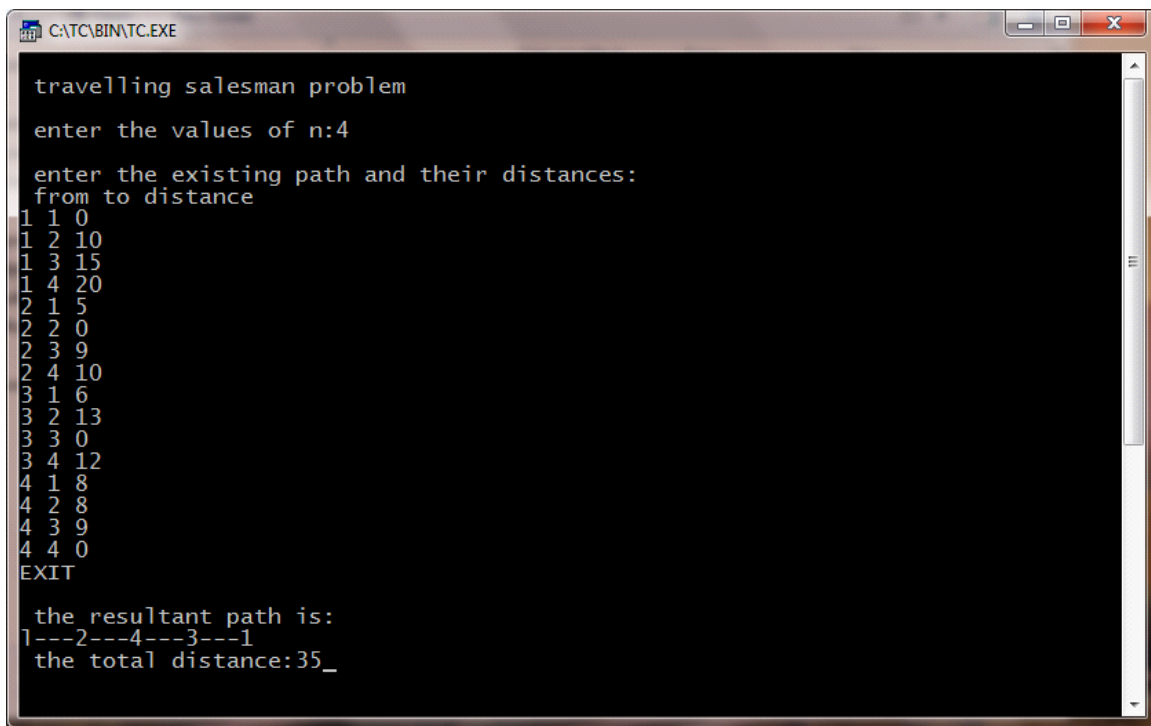
        }
    }
}
return(min);
}
void sales::display()
{
    cout<<"\nthe resultant path is:"<<endl;
    cout<<"1";
    for(i=0;i<n;i++)
    {
        cout<<"-----";
        cout<<result[i]+1;

    }
    cout<<"\n the total distance :"<<min;
}
int
main()
{
    sales t;
    t.input();
    t.display();

}

```

OUTPUT:



```
C:\TC\BIN\TC.EXE

travelling salesman problem
enter the values of n:4
enter the existing path and their distances:
from to distance
1 1 0
1 2 10
1 3 15
1 4 20
2 1 5
2 2 0
2 3 9
2 4 10
3 1 6
3 2 13
3 3 0
3 4 12
4 1 8
4 2 8
4 3 9
4 4 0
EXIT

the resultant path is:
1---2---4---3---1
the total distance:35_
```


**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

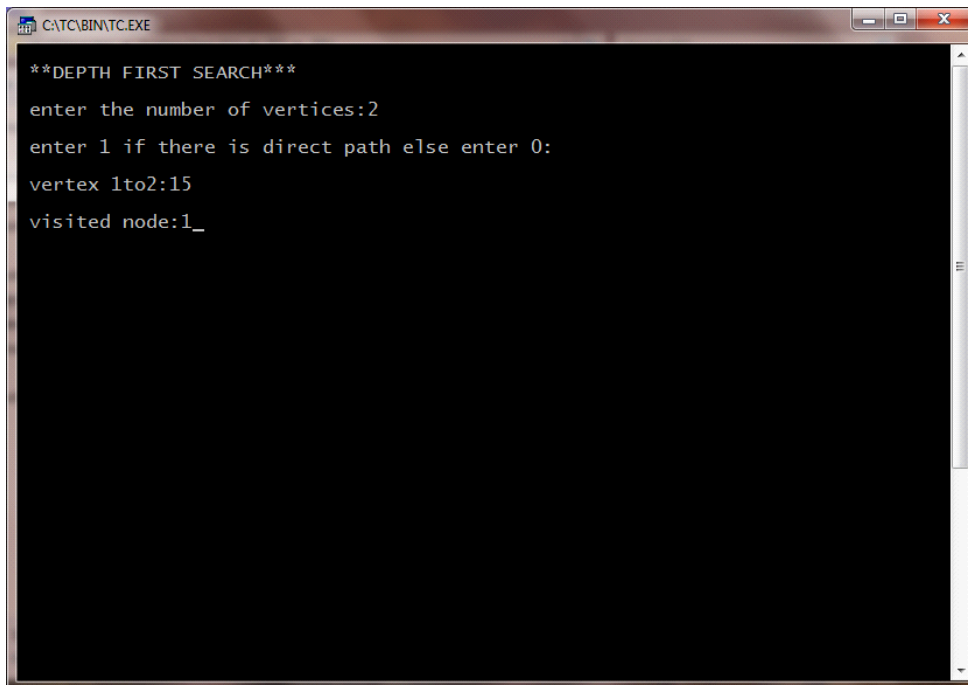
DEPTH FIRST SEARCH

```
#include<iostream>
using namespace std;
int n,mark[20],adj[50][50],i,j;
class dfs
{
    public:
    void getdata(void);
    void src(int n);
};
void dfs::src(int v)
{
    int w;
    mark[v]=1;
    cout<<"\nVisited node:"<<v;
    for(w=1;w<=n;w++)
        if((mark[w]==0)&&(adj[v][w]==1))src(w);
}
void dfs::getdata(void)
{
    cout<<"\nEnter the number of vertices:";
    cin>>n;
    cout<<"\nEnter if there is direct path else enter 0:\n";
    for(i=1;i<=n;i++)
    {
        for(j=i+1;j<=n;j++)
        {
            cout<<"\nvertex"<<i<<"to"<<j<<":";

```

```
        cin>>adj[i][j];
    }
}
int main()
{
    dfs d;
    d.getdata();
    for(i=1;i<=n;i++)
        mark[i]=0;
    d.src(1);
}
```

OUTPUT:



```
CATC\BIN\TC.EXE
**DEPTH FIRST SEARCH**
enter the number of vertices:2
enter 1 if there is direct path else enter 0:
vertex 1to2:15
visited node:1_
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

BREADTH FIRST SEARCH

```
#include<iostream>
using namespace std;
int i,j,n,a[50],mark[20],adj[50][50],top=0;
int del();
void add(int temp)
{
    a[++top]=temp;
}
int del()
{
    int temp;
    temp=a[1];
    for(i=1;i<top;i++)
        a[i]=a[i+1];
    top--;
    return(temp);
}
void bfs(int v)
{
    int w,u;
    mark[v]=1;
    add(v);
    while(top!=0)
    {
        u=del();
        cout<<"\n\nVISIT NODE="<<u;
```

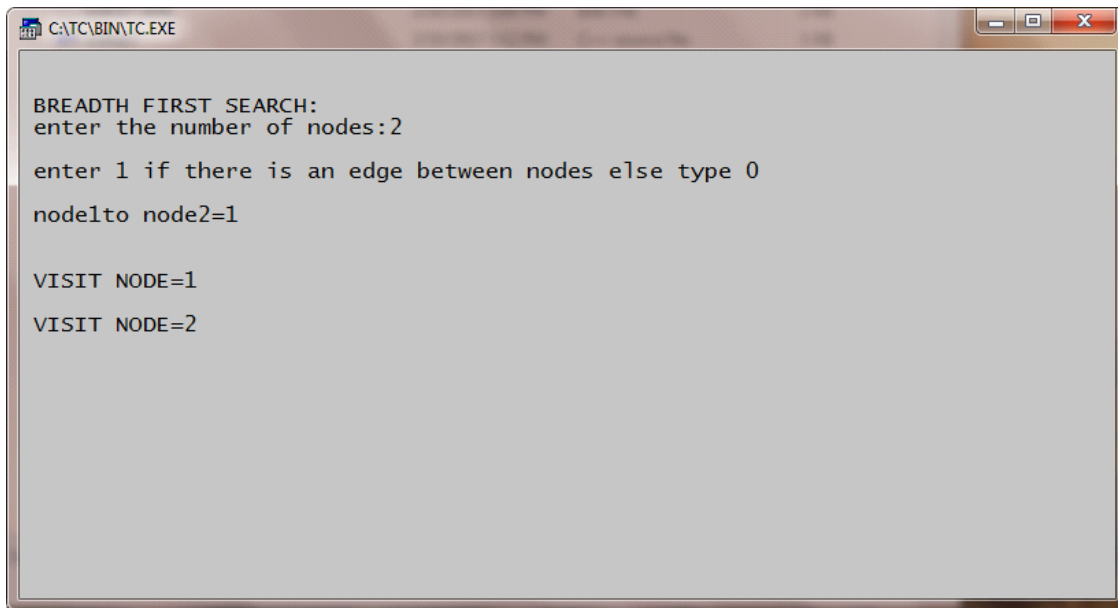
```

        for(w=1;w<n;w++)
        if((mark[w]==0)&&(adj[v][w]==1))
        {
            mark[w]=1;
            add(w);
        }
    }
}

int main()
{
    cout<<"\n\nBREATH FIRST SEARCH:";
    cout<<"\nEnter the no of nodes:";
    cin>>n;
    cout<<"\nEnter 1if there is an edge between nodes else type 0";
    for(i=1;i<=n;i++)
    {
        for(j=i+1;j<=n;j++)
        {
            cout<<"\n\n node"<<i<<"to node"<<j<<"=";
            cin>>adj[i][j];
        }
    }
    for(i=1;i<=n;i++)
    mark[i]=0;
    for(i=1;i<=n;i++)
    if(mark[i]==0)
    bfs(i);
}

```

OUTPUT:



```
C:\TC\BIN\TC.EXE

BREADTH FIRST SEARCH:
enter the number of nodes:2
enter 1 if there is an edge between nodes else type 0
node1to node2=1

VISIT NODE=1
VISIT NODE=2
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

N QUEENS

```
#include<iostream>
#include<stdlib.h>
using namespace std;
void addqueen();
int col[8],upfree[15],dnfree[15],coln[15];

void addqueen()
{
    int i,c,r;
    static int comb,row=-1;
    row++;
    for(i=0;i<=7;i++)
    {
        if(col[i]&&upfree[i+row]&&dnfree[row-i+7])
        {
            coln[row]=i;
            col[i]=0;
            upfree[i+row]=0;
            dnfree[row-1+7]=0;
            if(row>=7)
            {
                comb++;
                cout<<"\n\n\t 8 queens problem";
                cout<<"\n combination no:"<<comb;
                for(r=0;r<=7;r++)
```

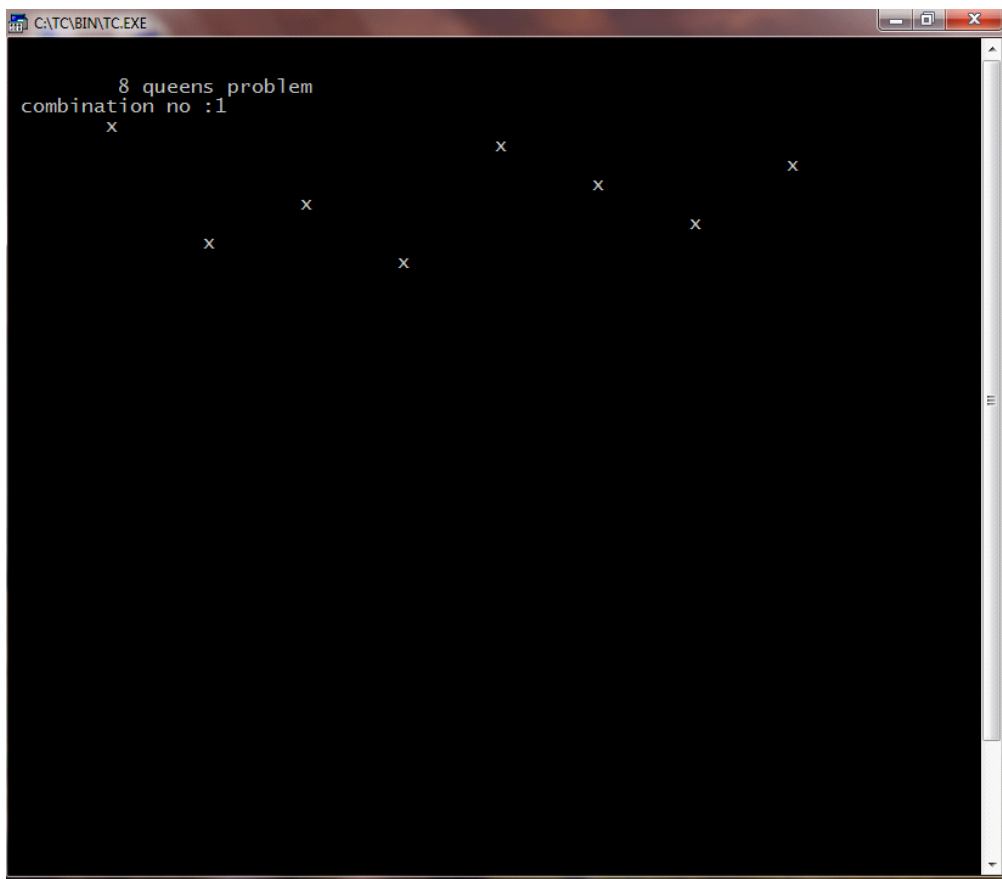
```

        {
            cout<<endl;
            for(c=0;c<=7;c++)
            {
                if(c=coln[r])
                {
                    cout<<"\tx";

                }
                else
                {
                    cout<<"\t";
                }
            }
        }
        else
        addqueen();
        col[coln[row]]=1;
        upfree[row+coln[row]]=1;
        dnfree[row-coln[row]+7]=1;
    }
}
row--;
}
int main()
{
    int i;
    for(i=0;i<=7;i++)
        col[i]=1;
    for(i=0;i<=14;i++)
        upfree[i]=dnfree[i]=1;
    addqueen();
}

```

OUTPUT:



A screenshot of a Windows command prompt window titled "C:\TC\BIN\TC.EXE". The window has a black background and displays the following text:

```
      8 queens problem  
combination no :1  
  x  
      x      x      x  
    x  x      x      x  
      x      x      x
```

The output shows a solution to the 8 queens problem, with queens placed at the following coordinates (row, column): (1,1), (2,3), (3,5), (4,7), (5,2), (6,4), and (7,6). The eighth queen is not visible, likely due to the window's dimensions.

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

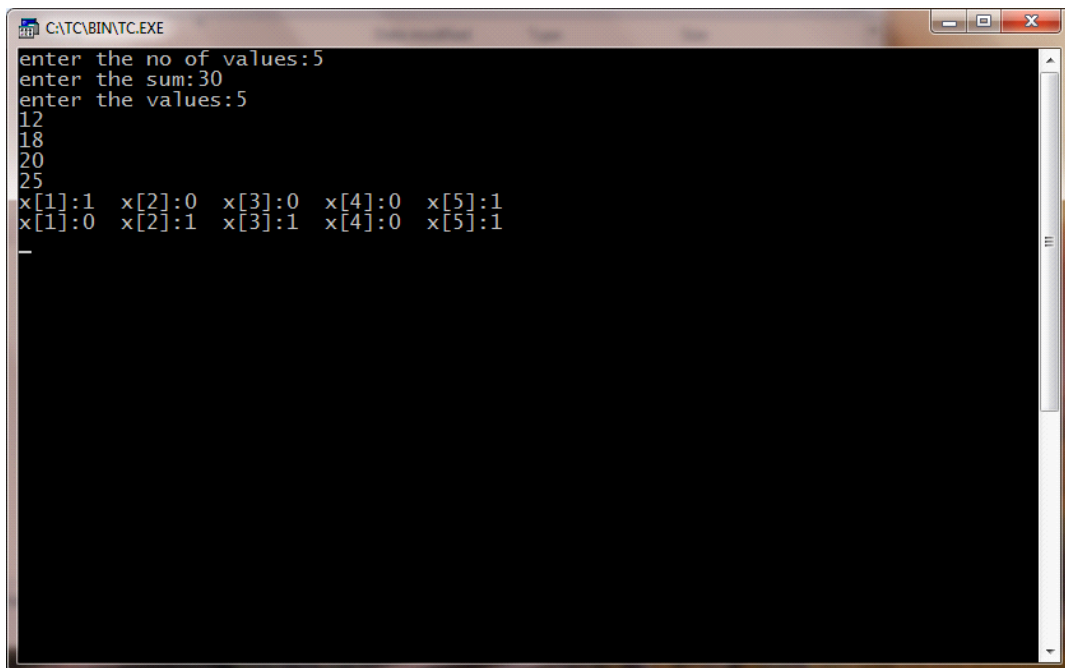
SUM OF SUBSET

```
#include<iostream.h>
#include<conio.h>
int m,n,w[10],x[10];
void sum_subset(int s,int k,int r)
{
    int i;
    x[k]=1;
    if(s+w[k]==m)
    {
        for(i=1;i<=n;i++)
        {
            cout<<"x["<<i<<"]:"<<x[i]<<"\t";
        }
        cout<<"\n";
    }
    else
    {
        if((s+w[k]+w[k+1])<=m)
        {
            sum_subset(s+w[k],k+1,r-w[k]);
        }
    }
    if(((s+r-w[k])>=m)&&((s+w[k+1])<=m))
    {
```

```
x[k]=0;
sum_subset(s,k+1,r-w[k]);
}
}
void main()
{
cout<<"enter the no of values:";
cin>>n;
cout<<"enter the sum:";
cin>>m;
cout<<"enter the values:";
int i;
for(i=1;i<=n;i++)
{
cin>>w[i];
}
int r;
```

```
r=0;
for(i=1;i<=n;i++)
{
r+=w[i];
}
sum_subset(0,1,r);
getch();
}
```

OUTPUT:



```
enter the no of values:5
enter the sum:30
enter the values:5
12
18
20
25
x[1]:1 x[2]:0 x[3]:0 x[4]:0 x[5]:1
x[1]:0 x[2]:1 x[3]:1 x[4]:0 x[5]:1
_
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

0/1 KNAPSACK PROBLEM USING BRANCH AND BOUND TECHNIQUE

```
#include<iostream.h>
#include<conio.h>
#include<stdlib.h>
#include<stdio.h>
int entries;
float ratio[20];
float *knapsack(int weight[20],int,int);
void sortpw(int pro[20],int weight[],int,int);
void main()
{
int pro[20];
int weight[20];
int maxwt=0,i,entries=0,count=0,minmax;
clrscr();
cout<<"\n enter no of entries:";
cin>>entries;
for(i=0;i<entries;i++)
{
cout<<"\n entry"<<i+1<<". ";
```

```

cout<<"\n enter profit:";
cin>>pro[i];
cout<<"\n enter weight[i]";
cin>>weight[i];
ratio[i]=(float)pro[i]/weight[i];
count++;
}
cout<<"maximum weight allowed:\n";
cin>>maxwt;
cout<<"\n maximum profit-->1 or 0:";
cin>>minmax;
float* wrat=new float[count];
sortpw(pro,weight,count,minmax);
wrat=knapsack(weight,maxwt,count);
cout<<"\nprofit\tweight\tratio\n";
for(i=0;i<count;i++)
cout<<pro[i]<<"\t"<<weight[i]<<"\t"<<wrat[i]<<"\n";
cout<<endl;
int totpro=0;
for(i=0;i<count;i++)
totpro+=pro[i]*wrat[i];
cout<<"\n total profit:"<<totpro;
getch();
}
void sortpw(int pro[],int weight[],int count,int minmax)
{
int i,j;
for(i=0;i<count;i++)
{
for(j=i+1;j<count;j++)
{
if((ratio[i]<ratio[j])&&minmax)
{
float t;
int tl;
t=ratio[i];
ratio[i]=ratio[j];
ratio[j]=t;
tl=pro[i];
pro[i]=pro[j];
pro[j]=tl;
tl=weight[i];
weight[i]=weight[j];
weight[j]=tl;
}
else if((ratio[i]>ratio[j])&&!minmax)

```

```

{
float t=0;
int tl;
t=ratio[i];
ratio[i]=ratio[j];
ratio[j]=t;
tl=pro[i];
pro[i]=pro[j];
pro[j]=tl;
tl=weight[i];
weight[i]=weight[j];
weight[j]=tl;
}
}
}
}

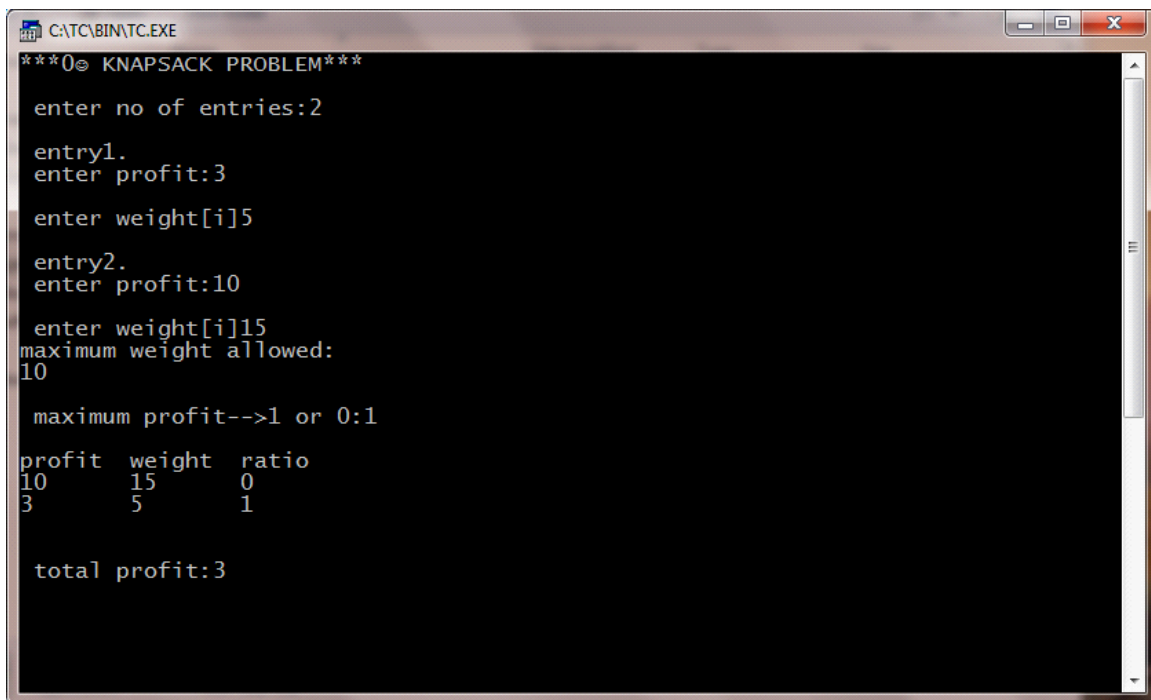
```

```

float *knapsack(int weight[20],int maxwt,int count)
{
int filledwt=0;
float *wratio=new float[count];
int i;
for(i=0;i<count;i++)
wratio[i]=0.0;
i=0;
while((filledwt<maxwt)&&(i<count))
{
if((filledwt+weight[i])<=maxwt)
{
wratio[i]=1.0;
filledwt+=weight[i];
}
i++;
if(i==count)
break;
}
return wratio;
}

```

OUTPUT:



```
C:\TC\BIN\TC.EXE
***0 KNAPSACK PROBLEM***

enter no of entries:2

entry1.
enter profit:3

enter weight[i]5

entry2.
enter profit:10

enter weight[i]15
maximum weight allowed:
10

maximum profit-->1 or 0:1

profit  weight  ratio
10      15      0
3       5       1

total profit:3
```

**NAME: MOHAMED B SIRAJUDEEN , REG.NO: 22TD0053 ,
YEAR/SEM/SEC: Y2/S4/B**

TRAVELLING SALESMAN PROBLEM

```
#include<iostream.h>
Using name space std;
int main()
{
int i,j,k,n,min,g[20][20],c[20][20],s,s1[20][1],s2,lb;
cout<<"\n TRAVELLING SALESMAN PROBLEM";
cout<<"\n input number of cities:";
cin>>n;
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
c[i][j]=0;
}
}
for(i=1;i<=n;i++)
{
for(j=1;j<=n;j++)
{
if(i==j)
continue;
else
{
cout<<"input"<<i<<"to"<<j<<"cost:";
cin>>c[i][j];
}
```



```

}
}
}
for(i=2;i<=n;i++)
{
g[i][0]=c[i][1];
}
for(i=2;i<=n;i++)
{
for(j=2;j<=n;j++)
{
if(i!=j)
g[i][j]=c[i][j]+g[j][0];
}
}
}

```

```

for(i=2;i<=n;i++)
{
for(j=2;j<=n;j++)
{
if(i!=j)
break;
}
}
for(k=2;k<=n;k++)
{
if(i!=k&&j!=k)
{
if((c[i][j]+g[i][k]<c[i][k]+g[k][j]))
{
g[i][j]=c[i][j]+g[j][k];
s1[i][j]=j;
}
else
{
g[i][1]=c[i][k]+g[k][j];
s1[i][1]=k;
}
}
}
}
min=c[1][2]+g[2][1];
s=2;
for(i=3;i<n;i++)
{
if((c[i][i]+g[i][i])<min)

```

```

{
min=c[1][i]+g[i][1];
s=i;
}
}
int y=g[i][1]+g[i][j]+g[i][i];
lb=y/2;
cout<<"edge matrix";
for(i=1;i<=n;i++)
{
cout<<"\n";
for(j=1;j<=n;j++)
{
cout<<"\t"<<c[i][j];
}
}

```

```

}
cout<<"\n min"<<min;

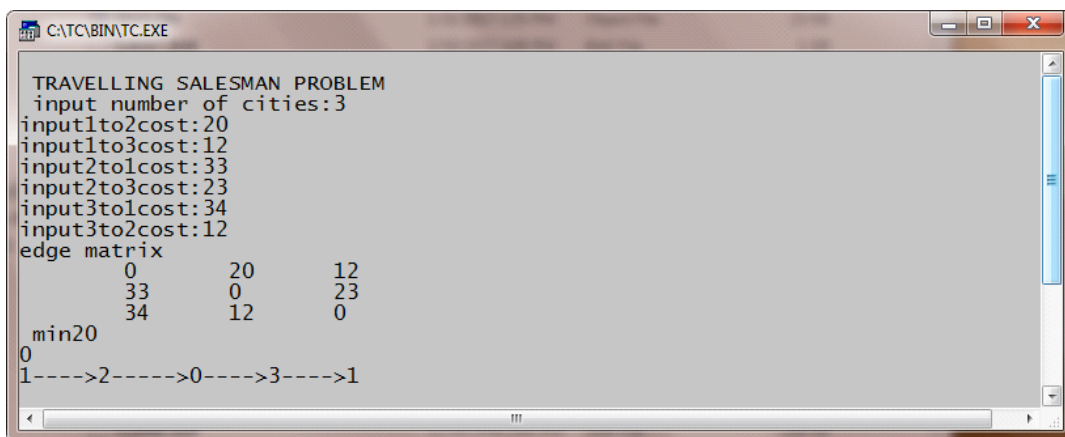
```

```

cout<<"\n\b"<<lb;
for(i=2;i<=n;i++)
{
if(s!=i&& s1[s][1]!=i)
{
s2=i;
}
}
cout<<"\n"<<1<<"---->"<<s<<"----->"<<s1[s][1]<<"---->"<<s2<<"----->"<<1<<"\n";
return 0;
}

```

OUTPUT:



```
TRAVELLING SALESMAN PROBLEM
input number of cities:3
input1to2cost:20
input1to3cost:12
input2to1cost:33
input2to3cost:23
input3to1cost:34
input3to2cost:12
edge matrix
      0      20      12
      33      0      23
      34      12      0
min20
0
1---->2---->0---->3---->1
```

